

Data Lab P1 实验报告

蔡雨桐 25800180009

题目要求

`signMask()` 函数要求返回一个掩码，该掩码中只有最高位（第 31 位）被置为 1，其余位全为 0。

- 目标值：0x80000000
- 可用运算符：! ~ & ^ | + << >>
- 最大操作数：2

代码实现

```
1 int signMask(void) {  
2     return 1 << 31;  
3 }
```

原理解析

在 32 位整数模型中，数据位按如下方式排列：

第 31 位	第 30 位	...	第 1 位	第 0 位	含义
1	0	...	0	0	二进制表示
2^{31}	2^{30}	...	2^1	2^0	对应的位权

- 数字 1 的二进制：0000 0000 0000 0000 0000 0000 0000 0001
- 左移 31 位 ($1 \ll 31$)：将 1 向左移动 31 位，低位补 0。
- 结果：1000 0000 0000 0000 0000 0000 0000 0000

这个二进制数转换为十六进制正好是 0x80000000。

操作数统计：代码只使用了 << 一个运算符，符合最大操作数不超过 2 的限制。

题目要求

bitXor(x, y) 函数要求实现异或 (XOR) 操作, 但只能使用 ~ 和 & 两种运算符。

- 示例: bitXor(4, 5) = 1, bitXor(7, 7) = 0
- 可用运算符: ! ~ & ^ | + << >> (但本题实际只能用 ~ 和 &)
- 最大操作数: 8

代码实现

```
1 int bitXor(int x, int y) {  
2     return ~(~x & ~y) & ~(x & y);  
3 }
```

原理解析

异或 (XOR) 的逻辑是: 两个数不同时为 1, 相同时为 0。其基本公式为:

$$x \oplus y = (x \& \sim y) \mid (\sim x \& y)$$

由于题目禁止使用 |, 我们需要利用**德摩根定律**进行转换:

$$A \mid B = \sim (\sim A \& \sim B)$$

我们采用另一种更简洁的推导方式:

$$\begin{aligned} x \oplus y &= (x \mid y) \& \sim (x \& y) \\ &= \sim (\sim x \& \sim y) \& \sim (x \& y) \end{aligned}$$

验证过程 (以 $x = 4, y = 5$ 为例):

- $x = 0100_2, y = 0101_2$
- $\sim x = 1011_2, \sim y = 1010_2$
- $\sim x \& \sim y = 1010_2$
- $\sim (\sim x \& \sim y) = 0101_2$
- $x \& y = 0100_2$
- $\sim (x \& y) = 1011_2$
- 最终结果: $0101_2 \& 1011_2 = 0001_2 = 1$

操作数统计: 代码共使用了 7 个运算符 (~ 3 次, & 4 次), 符合最大操作数不超过 8 的限制。

题目要求

`negativePart(x)` 函数要求：如果 $x < 0$ ，返回 $-x$ ；否则返回 0。

- 示例：`negativePart(-10) = 10`, `negativePart(5) = 0`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：6

代码实现

```
1 int negativePart(int x) {  
2     return (x >> 31) & (~x + 1);  
3 }
```

原理解析

第一步：提取符号掩码 $x \gg 31$

在 32 位补码表示中，最高位是符号位。算术右移 31 位后：

$$x \gg 31 = \begin{cases} 0x00000000 & \text{若 } x \geq 0 \\ 0xFFFFFFFF & \text{若 } x < 0 \end{cases}$$

这个结果正好可以作为“开关”使用。

第二步：计算相反数 $\sim x + 1$

在补码系统中，一个数的相反数定义为：

$$-x = \sim x + 1$$

例如 $x = -10$ ，其二进制补码为 $\dots 11110110$ ，则 $\sim x + 1 = \dots 00001010 = 10$ 。

第三步：组合

$$\text{result} = (x \gg 31) \& (\sim x + 1)$$

- 若 $x \geq 0$ ：掩码为 `0x00000000`，结果为 0。
- 若 $x < 0$ ：掩码为 `0xFFFFFFFF`，结果为 $\sim x + 1 = -x$ 。

操作数统计：共使用 4 个运算符 (`>>`, `~`, `+`, `&`)，符合最大操作数不超过 6 的限制。

题目要求

`copyByteWithin(x, src, dst)` 函数要求：将整数 x 的第 `src` 个字节 (0-3) 复制到第 `dst` 个字节，其他字节保持不变。

- 示例：`copyByteWithin(0x11223344, 0, 2) = 0x11443344`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：12

代码实现

```
1 int copyByteWithin(int x, int src, int dst) {
2     int srcShift = src << 3;
3     int dstShift = dst << 3;
4     int srcByte = (x >> srcShift) & 0xFF;
5     int clearMask = ~(0xFF << dstShift);
6     return (x & clearMask) | (srcByte << dstShift);
7 }
```

原理解析

第一步：计算位移量

每个字节占 8 位，因此第 k 个字节的起始位是 $k \times 8 = k \ll 3$ 。

$$\begin{aligned}\text{srcShift} &= \text{src} \ll 3 \\ \text{dstShift} &= \text{dst} \ll 3\end{aligned}$$

第二步：提取源字节

将 x 右移 `srcShift` 位，使目标字节落到最低 8 位，再用 `0xFF` 掩码提取：

$$\text{srcByte} = (x \gg \text{srcShift}) \& 0xFF$$

第三步：构造清零掩码

将 `0xFF` 左移 `dstShift` 位，得到只在目标字节位置为 1 的掩码；取反后得到只有目标字节位置为 0 的掩码：

$$\text{clearMask} = \sim (0xFF \ll \text{dstShift})$$

第四步：合并结果

先将 x 与 `clearMask` 按位与，清空目标字节；再将 `srcByte` 左移到目标位置，按位或合并：

$$\text{result} = (x \& \text{clearMask}) | (\text{srcByte} \ll \text{dstShift})$$

操作数统计：共使用 9 个运算符，符合最大操作数不超过 12 的限制。

题目要求

`logicalShift(x, n)` 函数要求：将 x 逻辑右移 n 位（高位补 0），其中 $0 \leq n \leq 31$ 。

- 示例：`logicalShift(0x87654321, 4) = 0x08765432`
- 可用运算符：! ~ & ^ | + << >>
- 最大操作数：20

代码实现

```
1 int logicalShift(int x, int n) {  
2     int shifted = x >> n;  
3     int mask = ~(((1 << 31) >> n) << 1);  
4     return shifted & mask;  
5 }
```

原理解析

问题背景

C 语言中的 `>>` 是**算术右移**：若 x 是负数，右移时高位会补 1。而题目要求的是**逻辑右移**，高位必须补 0。

第一步：算术右移

$$\text{shifted} = x \gg n$$

低 $32 - n$ 位是正确的，但高 n 位可能被补了 1。

第二步：构造掩码

$$\text{mask} = \sim (((1 \ll 31) \gg n) \ll 1)$$

拆解如下：

- $1 \ll 31 = 0x80000000$ （只有最高位是 1）。
- $(1 \ll 31) \gg n$ ：右移 n 位，高 n 位被补 1。
- $\ll 1$ ：左移 1 位，清除了最低位的 1，并保留了高 n 位的 1。
- $\sim$ ：取反，得到高 n 位为 0、其余位为 1 的掩码。

第三步：清除多余位

$$\text{result} = \text{shifted} \& \text{mask}$$

高 n 位被清零，低 $32 - n$ 位保持不变。

操作数统计：共使用 6 个运算符，符合最大操作数不超过 20 的限制。

题目要求

`swapNibblePairs(x)` 函数要求：交换整数 x 中**每个字节内部**的高 4 位 (nibble) 和低 4 位。

- 示例：`swapNibblePairs(0xAB) = 0xBA`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：18

代码实现

```
1 int swapNibblePairs(int x) {  
2     int low = (x & 0x0F0F0F0F) << 4;  
3     int high = (x >> 4) & 0x0F0F0F0F;  
4     return low | high;  
5 }
```

原理解析

第一步：提取所有低 4 位并移位

掩码 0x0F0F0F0F 的二进制形式为：

00001111 00001111 00001111 00001111

用它与 x 按位与，可以提取出每个字节的低 4 位，高 4 位被清零。然后左移 4 位，将其移到高 4 位的位置：

$\text{low} = (x \& 0x0F0F0F0F) \ll 4$

第二步：提取所有高 4 位并移位

先将 x 右移 4 位，让每个字节的高 4 位落到低 4 位的位置，再用掩码 0x0F0F0F0F 提取：

$\text{high} = (x \gg 4) \& 0x0F0F0F0F$

第三步：合并

用按位或合并两部分：

$\text{result} = \text{low} | \text{high}$

操作数统计：共使用 5 个运算符 ($\&$, $\ll$, $\gg$, $\&$, $|$)，符合最大操作数不超过 18 的限制。

题目要求

`secondLowestZeroBit(x)` 函数要求：返回一个掩码，该掩码中只有从最低位开始的第二个 0 位被置为 1。

- 示例：`secondLowestZeroBit(0xFFFFFFFFFA) = 0x4`, `secondLowestZeroBit(0x7FFFFFFF)` = 0, `secondLowestZeroBit(-1) = 0`
- 可用运算符：`!` `~` `&` `^` `|` `+` `<<` `>>`
- 最大操作数：8

代码实现

```
1 int secondLowestZeroBit(int x) {  
2     int y = x | (x + 1);  
3     return (~y & (y + 1)) & ~(y >> 31);  
4 }
```

原理解析

第一步：把最低位的 0 变成 1

$$y = x | (x + 1)$$

$x + 1$ 会把 x 中最低位的 0 变成 1，同时把它后面所有的 1 都变成 0。再与 x 按位或，就能把最低位的 0 置为 1。例如 $x = \dots 11111010$ ，最低位的 0 在第 0 位，则 $y = \dots 11111011$ 。

第二步：找 y 中最低位的 0

$$z = \sim y \& (y + 1)$$

- $y + 1$ 会把 y 中最低位的 0 变成 1，后面的 1 变成 0。
- $\sim y$ 会把 y 中最低位的 0 变成 1。
- 两者按位与，只有最低位的 0 的位置会保留为 1。

例如 $y = \dots 11111011$ ，最低位的 0 在第 2 位，则 $z = 0x4$ 。

第三步：处理边界情况

若 y 是全 1（即 $x = 0x7FFFFFFF$ 或 $x = -1$ ），则没有第二个 0。此时 $y \gg 31 = -1$ ， $\sim(y \gg 31) = 0$ ，结果为 0。

操作数统计：共使用 8 个运算符，符合最大操作数不超过 8 的限制。

题目要求

`oddParity(x)` 函数要求：判断 x 的二进制表示中 1 的个数是奇数还是偶数。如果是偶数返回 1，如果是奇数返回 0。

- 示例: $\text{oddParity}(5) = 1$, $\text{oddParity}(7) = 0$
- 可用运算符: `! ~ & ^ | + << >>`
- 最大操作数: 56

代码实现

```

1 int oddParity(int x) {
2     int y = x ^ (x >> 16);
3     y = y ^ (y >> 8);
4     y = y ^ (y >> 4);
5     y = y ^ (y >> 2);
6     y = y ^ (y >> 1);
7     return !(y & 1);
8 }

```

原理解析

异或折叠的核心思想

异或运算具有“模 2 加法”的特性： $a \oplus b$ 相当于把两个数对应位相加（不进位）。通过不断折叠，可以将所有位的奇偶性压缩到最低位。

折叠过程

$$y_1 = x \oplus (x \gg 16) \quad (32 \text{ 位} \rightarrow 16 \text{ 位})$$

$$y_2 = y_1 \oplus (y_1 \gg 8) \quad (16 \text{ 位} \rightarrow 8 \text{ 位})$$

$$y_3 = y_2 \oplus (y_2 \gg 4) \quad (8 \text{ 位} \rightarrow 4 \text{ 位})$$

$$y_4 = y_3 \oplus (y_3 \gg 2) \quad (4 \text{ 位} \rightarrow 2 \text{ 位})$$

$$y_5 = y_4 \oplus (y_4 \gg 1) \quad (2 \text{ 位} \rightarrow 1 \text{ 位})$$

最终 y_5 的最低位即为所有 32 位中 1 的个数的奇偶性。

最终判断

$$\text{result} = \sim (y_5 \& 1)$$

- 若 1 的个数为偶数, $y_5 \& 1 = 0$, 返回 1。
- 若 1 的个数为奇数, $y_5 \& 1 = 1$, 返回 0。

操作数统计：共使用 12 个运算符（5 次 $\gg$ ，5 次 $\wedge$ ，1 次 $\&$ ，1 次 $!$ ），符合最大操作数不超过 56 的限制。

题目要求

`rotateRightBits(x, n)` 函数要求：将 x 循环右移 n 位，其中 $n \geq 0$ ，且实际移动位数按 32 取模。

- 示例：`rotateRightBits(0x12345678, 8) = 0x78123456`
- 可用运算符： $! \sim \& \wedge | + \ll \gg$
- 最大操作数：16

代码实现

```
1 int rotateRightBits(int x, int n) {
2     n = n & 31;
3     int left = (32 + ~n + 1) & 31;
4     int right = x >> n;
5     int mask = ~(((1 << 31) >> n) << 1);
6     return (right & mask) | (x << left);
7 }
```

原理解析

循环右移的数学公式

$$\text{rotateRight}(x, n) = (x \gg n) | (x \ll (32 - n))$$

两个陷阱

1. $x \gg n$ 是**算术右移**：若 x 是负数，高位会补 1，但循环移位应补 0（因为移出的位会从左边回来）。
2. $x \ll (32 - n)$ 在 $n = 0$ 时会变成 $x \ll 32$ ，这是**未定义行为**。

代码逐行解析

第一步：对 n 取模 32

$$n = n \& 31$$

循环移位周期为 32，移 32 位等于没移。

第二步：计算左移量

$$\text{left} = (32 + \sim n + 1) \& 31 = (32 - n) \& 31$$

当 $n = 0$ 时， $\text{left} = 0$ ，避免移位 32 位。

第三步：算术右移

$$\text{right} = x \gg n$$

低 $32 - n$ 位正确，高 n 位可能被补 1。

第四步：构造掩码

$$\text{mask} = \sim (((1 \ll 31) \gg n) \ll 1)$$

清除算术右移补的高位 1。

第五步：合并

$$\text{result} = (\text{right} \& \text{mask}) | (x \ll \text{left})$$

操作数统计：共使用 13 个运算符，符合最大操作数不超过 16 的限制。

题目要求

`roundEvenPow2(x, n)` 函数要求：将非负整数 x 舍入到最接近的 2^n 的倍数。若 x 恰好在两个倍数正中间，则向“商为偶数”的方向舍入。

- 示例：`roundEvenPow2(10, 2) = 8`，`roundEvenPow2(14, 2) = 16`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：24

代码实现

```
1 int roundEvenPow2(int x, int n) {  
2     int mask = (1 << n) + ~0;  
3     int half = 1 << (n + ~0);
```

```

4     int rem = x & mask;
5     int down = x & ~mask;
6     int up = (x + half) & ~mask;
7
8     int diff = rem + ~half + 1;
9     int gtHalf = !(diff & ~(diff >> 31));
10    int isHalf = !diff;
11    int upEven = !((up >> n) & 1);
12    int useUp = gtHalf | (isHalf & upEven);
13
14    int chooseUp = ~useUp + 1;
15    return (up & chooseUp) | (down & ~chooseUp);
16 }

```

原理解析

核心思路：四舍五入 + 中点偶数修正

第一步：计算基本量

$\text{mask} = (1 \ll n) + \sim 0$ (低 n 位全 1)

$\text{half} = 1 \ll (n - 1)$ (中点)

$\text{rem} = x \& \text{mask}$ (余数)

$\text{down} = x \& \sim \text{mask}$ (向下舍入)

$\text{up} = (x + \text{half}) \& \sim \text{mask}$ (向上舍入)

第二步：判断舍入方向

- 若 $\text{rem} > \text{half}$ ：选 up。
- 若 $\text{rem} < \text{half}$ ：选 down。
- 若 $\text{rem} = \text{half}$ ：看 $\text{up} \gg n$ 是否为偶数。若是偶数，选 up；否则选 down。

用位运算表示：

$\text{diff} = \text{rem} - \text{half}$

$\text{gtHalf} = \text{diff} > 0 ? 1 : 0$

$\text{isHalf} = \text{diff} == 0 ? 1 : 0$

$\text{upEven} = (\text{up} \gg n) \& 1 == 0 ? 1 : 0$

$\text{useUp} = \text{gtHalf} | (\text{isHalf} \& \text{upEven})$

第三步：用掩码选择结果

$\text{chooseUp} = \sim \text{useUp} + 1$

- $\text{useUp} = 1$ ： $\text{chooseUp} = 0xFFFFFFFF$ ，返回 up。

- `useUp = 0`: `chooseUp = 0`, 返回 `down`。

操作数统计：共使用约 20 个运算符，符合最大操作数不超过 24 的限制。

题目要求

`midpointTowardFirst(x, y)` 函数要求：计算 $(x + y)/2$ ，要求不能溢出，且当结果位于两个整数正中间时，向第一个参数 x 的方向取整。

- 示例：`midpointTowardFirst(4, 7) = 5`, `midpointTowardFirst(7, 4) = 6`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：32

代码实现

```
1 int midpointTowardFirst(int x, int y) {
2     int avg = (x & y) + ((x ^ y) >> 1);
3     int lost = (x ^ y) & 1;
4
5     int sign_x = x >> 31;
6     int sign_y = y >> 31;
7     int diff = x + ~y + 1;
8     int sign_diff = diff >> 31;
9
10    int x_gt_y = (~sign_x & sign_y) | ((~(sign_x ^ sign_y)) & ~sign_diff & !!diff);
11
12    int roundup = lost & x_gt_y;
13    return avg + roundup;
14 }
```

原理解析

第一步：无溢出平均

直接 $(x + y) \gg 1$ 可能溢出。安全公式为：

$$\text{avg} = (x \& y) + ((x \oplus y) \gg 1)$$

该公式默认向负无穷取整（对正数相当于向下取整）。

第二步：判断是否需要修正

$$\text{lost} = (x \oplus y) \& 1$$

若 x 和 y 奇偶性不同，则精确中点为 .5，需要向 x 方向修正。

第三步：判断 $x > y$ （避免溢出）

```
sign_x = x >> 31
sign_y = y >> 31
diff = x - y
sign_diff = diff >> 31
```

判断条件：

$$\text{x_gt_y} = (\sim \text{sign_x} \& \text{sign_y}) \mid (\sim (\text{sign_x} \oplus \text{sign_y}) \& \sim \text{sign_diff} \& !!\text{diff})$$

- $\sim \text{sign_x} \& \text{sign_y}$: x 非负且 y 负数。
- 同号时用减法结果判断。

第四步：修正

```
roundup = lost & x_gt_y
result = avg + roundup
```

操作数统计：共使用 24 个运算符，符合最大操作数不超过 32 的限制。

题目要求

`isBetweenEitherOrder(x, a, b)` 函数要求：判断 x 是否在闭区间 $[a, b]$ 或 $[b, a]$ 内（端点顺序任意）。在则返回 1，否则返回 0。

- 示例：`isBetweenEitherOrder(5, 8, 3) = 1`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：48

代码实现

```
1 int isBetweenEitherOrder(int x, int a, int b) {
2     int sa = a >> 31;
3     int sb = b >> 31;
4     int same_sign = ~(sa ^ sb);
5     int diff_ab = a + ~b + 1;
6     int sdiff_ab = diff_ab >> 31;
7     int mask = (same_sign & sdiff_ab) | (~same_sign & sa);
8
9     int min = (a & mask) | (b & ~mask);
10    int max = (b & mask) | (a & ~mask);
11
12    int sx = x >> 31;
13    int smin = min >> 31;
14    int diff_min = x + ~min + 1;
15    int sdiff_min = diff_min >> 31;
16    int same_sign_min = ~(sx ^ smin);
17    int x_ge_min = (same_sign_min & ~sdiff_min) | (~same_sign_min & ~sx);
18
19    int smax = max >> 31;
20    int diff_max = max + ~x + 1;
21    int sdiff_max = diff_max >> 31;
22    int same_sign_max = ~(smax ^ sx);
23    int max_ge_x = (same_sign_max & ~sdiff_max) | (~same_sign_max & ~smax);
24
25    return !(x_ge_min & max_ge_x);
26 }
```

原理解析

溢出安全的大小比较

直接使用减法 $a - b$ 判断大小可能会溢出（例如 $a = \text{INT_MAX}, b = -1$ ）。解决方案是：先判断两个数的符号是否相同。

- 同号：减法不会溢出，直接用差的符号位判断。
- 异号：减法可能溢出，直接看被减数的符号位。

第一步：判断 $a < b$

```
same_sign = ~ (sa ⊕ sb)
sdiff_ab = (a - b) >> 31
mask = (same_sign & sdiff_ab) | (~ same_sign & sa)
```

第二步：确定 min 和 max

```
min = (a & mask) | (b & ~ mask)
max = (b & mask) | (a & ~ mask)
```

第三步、第四步：判断 x 是否在区间内

用同样的溢出安全方法判断 $x \geq \text{min}$ 和 $\text{max} \geq x$ 。

第五步：合并

```
result = !(x_ge_min & max_ge_x)
```

操作数统计：共使用约 44 个运算符，符合最大操作数不超过 48 的限制。

题目要求

`mul5Sat(x)` 函数要求：计算 $x \times 5$ 。如果发生溢出，正数溢出返回 `INT_MAX`，负数溢出返回 `INT_MIN`。

- 示例：`mul5Sat(1) = 5`, `mul5Sat(0x40000000) = 0x7FFFFFFF`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：30

代码实现

```
1 int mul5Sat(int x) {
2     int mul5 = (x << 2) + x;
3     int sign_x = x >> 31;
4
5     int lim_pos = 0x19999999;
6     int lim_neg = ~0x19999999 + 1;
7
8     int pos_check = (x + ~lim_pos) >> 31;
9     int pos_ovf = ~pos_check & ~sign_x;
10
11    int neg_check = (x + ~lim_neg + 1) >> 31;
12    int neg_ovf = neg_check & sign_x;
13
14    int tmin = 1 << 31;
15    int tmax = ~tmin;
16
17    int no_ovf_mask = ~(pos_ovf | neg_ovf);
18
19    int sat_pos = tmax & pos_ovf;
20    int sat_neg = tmin & neg_ovf;
21
22    return (mul5 & no_ovf_mask) | sat_pos | sat_neg;
23 }
```

原理解析

第一步：计算 $x \times 5$

$$\text{mul5} = (x \ll 2) + x$$

第二步：预判溢出边界

$x \times 5$ 不溢出的条件是：

$$\text{INT_MIN}/5 \leq x \leq \text{INT_MAX}/5$$

即：

$$-0x19999999 \leq x \leq 0x19999999$$

第三步：判断正溢出

```
pos_check = (x - lim_pos) >> 31
pos_ovf = ~pos_check & ~sign_x
```

第四步：判断负溢出

```
neg_check = (x - lim_neg) >> 31
neg_ovf = neg_check & sign_x
```

第五步：构造饱和值和掩码

```
tmin = 1 << 31
tmax = ~tmin
no_ovf_mask = ~(pos_ovf | neg_ovf)
sat_pos = tmax & pos_ovf
sat_neg = tmin & neg_ovf
```

第六步：组合结果

```
result = (mul5 & no_ovf_mask) | sat_pos | sat_neg
```

操作数统计：共使用 23 个运算符，符合最大操作数不超过 30 的限制。

题目要求

classifyAdd3(x, y, z) 函数要求：判断 $x+y+z$ 是否超出 32 位补码范围。若大于 INT_MAX 返回 1，若小于 INT_MIN 返回 -1，否则返回 0。

- 可用运算符: ! ~ & ^ | + << >>
- 最大操作数: 52

代码实现

```

1 int classifyAdd3(int x, int y, int z) {
2     int sum1 = x + y;
3     int sign_x = x >> 31;
4     int sign_y = y >> 31;
5     int sign_z = z >> 31;
6     int sign_sum1 = sum1 >> 31;
7
8     int pos_ovf1 = ~sign_x & ~sign_y & sign_sum1;
9     int neg_ovf1 = sign_x & sign_y & ~sign_sum1;
10
11     int sum2 = sum1 + z;
12     int sign_sum2 = sum2 >> 31;
13
14     int pos_ovf2 = ~sign_sum1 & ~sign_z & sign_sum2;
15     int neg_ovf2 = sign_sum1 & sign_z & ~sign_sum2;
16
17     int any_pos_ovf = pos_ovf1 | pos_ovf2;
18     int any_neg_ovf = neg_ovf1 | neg_ovf2;
19
20     return (any_pos_ovf & 1) + (any_neg_ovf & -1);
21 }

```

原理解析

第一步: 计算 $x + y$ 并检测溢出

$$\begin{aligned} \text{pos_ovf1} &= \sim \text{sign_x} \& \sim \text{sign_y} \& \text{sign_sum1} \\ \text{neg_ovf1} &= \text{sign_x} \& \text{sign_y} \& \sim \text{sign_sum1} \end{aligned}$$

- 正溢出: x, y 同正, 但 sum1 变负。
- 负溢出: x, y 同负, 但 sum1 变正。

第二步: 计算 $\text{sum1} + z$ 并检测溢出

$$\begin{aligned} \text{pos_ovf2} &= \sim \text{sign_sum1} \& \sim \text{sign_z} \& \text{sign_sum2} \\ \text{neg_ovf2} &= \text{sign_sum1} \& \text{sign_z} \& \sim \text{sign_sum2} \end{aligned}$$

第三步：合并结果

$$\text{result} = (\text{any_pos_ovf} \& 1) + (\text{any_neg_ovf} \& -1)$$

使用加法而不是按位或，避免 $1 \mid -1 = -1$ 的错误。

操作数统计：共使用约 25 个运算符，符合最大操作数不超过 52 的限制。

题目要求

`floatScaleThreeHalves(uf)` 函数要求：对单精度浮点数 f （以 `unsigned` 位模式传入）计算 $f \times 3/2$ ，返回结果的位模式。

- 使用 round-to-nearest-even 舍入。
- 保留 +0 和 -0 的符号。
- NaN 返回自身。
- 可用运算符：任意整数/无符号运算，允许 `if`、`while` 等控制流。
- 最大操作数：60

代码实现

```
1 unsigned floatScaleThreeHalves(unsigned uf) {
2     unsigned sign = uf >> 31;
3     unsigned exp = (uf >> 23) & 0xFF;
4     unsigned frac = uf & 0x7FFFFFFF;
5
6     // 处理 NaN 和无穷大
7     if (exp == 0xFF) return uf;
8     // 处理 +0 和 -0
9     if (exp == 0 && frac == 0) return uf;
10
11    // 处理非规格化数 (exp == 0)
12    if (exp == 0) {
13        unsigned temp = frac * 3;
14        unsigned new_frac = temp >> 1;
15        unsigned remainder = temp & 1;
16    }
```

```

17     // 向偶数舍入
18     if (remainder && (new_frac & 1)) {
19         new_frac++;
20     }
21
22     // 检查进位
23     if (new_frac & 0x800000) {
24         exp = 1;
25         new_frac &= 0x7FFFFFFF;
26     }
27     return (sign << 31) | (exp << 23) | new_frac;
28 }
29
30 // 处理规格化数 (exp != 0)
31 unsigned m = 0x800000 | frac;
32 unsigned temp = m * 3;
33 unsigned new_m = temp >> 1;
34 unsigned remainder = temp & 1;
35
36 // 向偶数舍入
37 if (remainder && (new_m & 1)) {
38     new_m++;
39 }
40 // 检查是否溢出到高位 (进位到阶码)
41 if (new_m & 0x1000000) {
42     remainder = new_m & 1;
43     new_m >>= 1;
44     if (remainder && (new_m & 1)) {
45         new_m++;
46     }
47     exp++;
48 }
49
50 // 处理阶码溢出为无穷大
51 if (exp >= 0xFF) {
52     return (sign << 31) | 0x7F800000;
53 }
54
55 unsigned new_frac = new_m & 0x7FFFFFFF;
56 return (sign << 31) | (exp << 23) | new_frac;
57 }

```

原理解析

浮点数结构

单精度浮点数由 32 位组成：

- 第 31 位：符号位 sign

- 第 30-23 位：指数 `exp` (8 位，偏移量 127)
- 第 22-0 位：尾数 `frac` (23 位)

非规格化数的处理

非规格化数的值是 $f = \text{frac} \times 2^{-149}$ 。乘以 1.5 后：

$$f \times 1.5 = \text{frac} \times 3 \times 2^{-150}$$

新的尾数为 $(\text{frac} \times 3) \gg 1$ 。

规格化数的处理

规格化数的值是 $f = (1 + \text{frac} \times 2^{-23}) \times 2^{\text{exp}-127}$ 。令 $m = 2^{23} + \text{frac}$ ，则：

$$f \times 1.5 = (m \times 3 \times 2^{-24}) \times 2^{\text{exp}-127}$$

新的尾数为 $(m \times 3) \gg 1$ 。

向偶数舍入

若余数为 1 (有 .5) 且结果尾数是奇数，则进位到偶数。

操作数统计：共使用约 35 个运算符，符合最大操作数不超过 60 的限制。

Data Lab P16: floatRoundEven 解析

题目要求

`floatRoundEven(uf)` 函数要求：将单精度浮点数 uf 舍入到最接近的整数，采用 round-to-nearest-even 规则，返回该整数的单精度浮点位模式。

- 若舍入结果为 0，保留输入符号。
- NaN 或无穷大返回自身。
- 可用运算符：任意整数/无符号运算，允许 `if`、`while` 等控制流。
- 最大操作数：65

代码实现

```
1 unsigned floatRoundEven(unsigned uf) {
2     unsigned sign = uf >> 31;
3     unsigned exp = (uf >> 23) & 0xFF;
4     unsigned frac = uf & 0x7FFFFFFF;
5
6     // 1. NaN 或 Infinity 直接返回
7     if (exp == 0xFF) return uf;
8
9     // 2. 绝对值小于 1.0
10    if (exp < 127) {
11        if (exp < 126) return sign << 31;
12        if (exp == 126 && frac == 0) return sign << 31;
13        return (sign << 31) | (127 << 23);
14    }
15
16    int shift = 23 - (exp - 127);
17    if (shift <= 0) return uf;
18
19    // 3. 提取整数部分和小数部分
20    unsigned full_frac = (1 << 23) | frac;
21    unsigned int_part = full_frac >> shift;
22    unsigned mask = (shift >= 32) ? 0xFFFFFFFF : ((1U << shift) - 1);
23    unsigned frac_part = full_frac & mask;
24    unsigned half = 1U << (shift - 1);
25
26    unsigned round_up = 0;
27    if (frac_part > half) {
28        round_up = 1;
29    } else if (frac_part == half) {
30        if (int_part & 1) round_up = 1;
31    }
32
33    unsigned rounded_val = int_part + round_up;
34    if (rounded_val == 0) return sign << 31;
35
36    // 4. 将舍入后的整数重新转换为浮点数
37    unsigned temp = rounded_val;
38    unsigned msb = 0;
39    while (temp > 1) {
40        temp >>= 1;
41        msb++;
42    }
43
44    unsigned res_exp = 127 + msb;
45    unsigned res_frac;
46    if (msb >= 23) {
47        res_frac = (rounded_val >> (msb - 23)) & 0x7FFFFFFF;
```

```

48     } else {
49         res_frac = (rounded_val << (23 - msb)) & 0x7FFFFFFF;
50     }
51
52     // 5. 检查指数溢出
53     if (res_exp >= 0xFF) {
54         return (sign << 31) | (0xFF << 23);
55     }
56
57     return (sign << 31) | (res_exp << 23) | res_frac;
58 }

```

原理解析

浮点数结构

单精度浮点数由 32 位组成：

- 第 31 位：符号位 **sign**
- 第 30-23 位：指数 **exp**
- 第 22-0 位：尾数 **frac**

分类处理

1. $|\text{exp}| = 0xFF$ ：NaN 或无穷大，返回自身。
2. $\text{exp} < 127$ ： $|f| < 1.0$ 。
 - $\text{exp} < 126$ ： $|f| < 0.5$ ，舍入到 0。
 - $\text{exp} = 126$ 且 $\text{frac} = 0$ ： $|f| = 0.5$ ，向偶数舍入到 0。
 - 否则： $0.5 < |f| < 1$ ，舍入到 ± 1 。
3. $\text{exp} \geq 127$ ：提取整数和小数部分，向偶数舍入。

向偶数舍入规则

- 小数部分 $>$ 中点：向上舍入。
- 小数部分 $<$ 中点：向下舍入。
- 小数部分 $=$ 中点：若整数部分为奇数，向上舍入；否则保持不变。

示例验证

以 $uf = 0x3fc00000$ (1.5) 为例:

- `exp = 127, frac = 0x400000`
- `shift = 23, int_part = 1, frac_part = 0x400000`
- `frac_part = half, int_part` 为奇数, `round_up = 1`
- `rounded_val = 2`, 重新编码为 `0x40000000` (2.0)

操作数统计: 共使用约 45 个运算符, 符合最大操作数不超过 65 的限制。

题目要求

`float_i2f(x)` 函数要求: 将整数 x 转换为单精度浮点数 (返回 `unsigned` 位模式), 使用 `round-to-nearest-even` 舍入。

- **可用运算符:** 任意整数/无符号运算, 允许 `if`、`while` 等控制流。
- **最大操作数:** 40

代码实现

```
1 unsigned float_i2f(int x) {
2     if (x == 0) return 0;
3
4     unsigned sign = x >> 31;
5     unsigned abs_x = x;
6     if (sign) abs_x = -x;
7
8     int highest_bit = 31;
9     while (!(abs_x & (1 << highest_bit))) highest_bit--;
10
11     unsigned exp = 127 + highest_bit;
12
13     unsigned frac;
14     if (highest_bit <= 23) {
15         frac = (abs_x << (23 - highest_bit)) & 0x7FFFFFFF;
16     } else {
17         int shift = highest_bit - 23;
```

```

18     frac = (abs_x >> shift) & 0x7FFFFFFF;
19
20     unsigned dropped = abs_x & ((1 << shift) - 1);
21     unsigned half = 1 << (shift - 1);
22
23     int round_up = 0;
24     if (dropped > half) {
25         round_up = 1;
26     } else if (dropped == half) {
27         if (frac & 1) round_up = 1;
28     }
29
30     if (round_up) {
31         frac++;
32         if (frac & 0x800000) {
33             frac = 0;
34             exp++;
35         }
36     }
37 }
38
39 return (sign << 31) | (exp << 23) | (frac & 0x7FFFFFFF);
40 }

```

原理解析

浮点数结构

单精度浮点数由 32 位组成：

- 第 31 位：符号位 `sign`
- 第 30-23 位：指数 `exp`（偏移量 127）
- 第 22-0 位：尾数 `frac`

处理步骤

1. 若 $x = 0$ ，直接返回 0。
2. 提取符号并取绝对值。
3. 找到最高位的 1 的位置 `highest_bit`。
4. 计算指数 $\text{exp} = 127 + \text{highest_bit}$ 。
5. 构造尾数：
 - 若 $\text{highest_bit} \leq 23$ ：直接左移。

- 若 `highest_bit > 23`: 右移并舍入。

6. 组装结果。

向偶数舍入

- 被丢弃的位 $>$ 中点: 向上舍入。
- 被丢弃的位 $<$ 中点: 向下舍入。
- 被丢弃的位 $=$ 中点: 若尾数为奇数, 向上舍入。

操作数统计: 共使用约 35 个运算符, 符合最大操作数不超过 40 的限制。

题目要求

bitCount(x) 函数要求：计算整数 x 的二进制表示中 1 的个数。

- 示例：bitCount(5) = 2, bitCount(7) = 3
- 可用运算符：! ~ & ^ | + << >>
- 最大操作数：40

代码实现

```
1 int bitCount(int x) {
2     int mask1 = 0x55 | (0x55 << 8);
3     mask1 = mask1 | (mask1 << 16);
4     int mask2 = 0x33 | (0x33 << 8);
5     mask2 = mask2 | (mask2 << 16);
6     int mask4 = 0x0F | (0x0F << 8);
7     mask4 = mask4 | (mask4 << 16);
8     int mask8 = 0xFF | (0xFF << 16);
9     int mask16 = 0xFF | (0xFF << 8);
10
11     int count = (x & mask1) + ((x >> 1) & mask1);
12     count = (count & mask2) + ((count >> 2) & mask2);
13     count = (count & mask4) + ((count >> 4) & mask4);
14     count = (count & mask8) + ((count >> 8) & mask8);
15     count = (count & mask16) + ((count >> 16) & mask16);
16
17     return count;
18 }
```

原理解析

并行加法思想

将 32 位分成若干个小组，先计算每组中 1 的个数，再逐层合并相邻组的计数。

掩码构造

```
mask1 = 0x55555555 (每 2 位取低 1 位)
mask2 = 0x33333333 (每 4 位取低 2 位)
mask4 = 0x0F0F0F0F (每 8 位取低 4 位)
mask8 = 0x00FF00FF (每 16 位取低 8 位)
mask16 = 0x0000FFFF (每 32 位取低 16 位)
```

逐层合并

```
count = (x & mask1) + ((x >> 1) & mask1)
count = (count & mask2) + ((count >> 2) & mask2)
count = (count & mask4) + ((count >> 4) & mask4)
count = (count & mask8) + ((count >> 8) & mask8)
count = (count & mask16) + ((count >> 16) & mask16)
```

示例验证

以 $x = 5 = 101_2$ 为例：

- 第 1 步：每 2 位一组，得到 $1 + 1 = 2$ 。
- 后续步骤合并后，最终 $\text{count} = 2$ 。

操作数统计：共使用约 37 个运算符，符合最大操作数不超过 40 的限制。

题目要求

`bitReverse(x)` 函数要求：将一个 32 位整数的二进制位完全翻转（第 0 位变第 31 位，第 1 位变第 30 位，以此类推）。

- 示例：`bitReverse(0x80000004) = 0x20000001`
- 可用运算符：`! ~ & ^ | + << >>`
- 最大操作数：34

代码实现

```
1 int bitReverse(int x) {  
2     x = ((x >> 16) & 0x0000FFFF) | ((x & 0x0000FFFF) << 16);  
3     x = ((x >> 8) & 0x00FF00FF) | ((x & 0x00FF00FF) << 8);  
4     x = ((x >> 4) & 0x0F0F0F0F) | ((x & 0x0F0F0F0F) << 4);  
5     x = ((x >> 2) & 0x33333333) | ((x & 0x33333333) << 2);  
6     x = ((x >> 1) & 0x55555555) | ((x & 0x55555555) << 1);  
7     return x;  
8 }
```

原理解析

分治交换思想

翻转 32 位可以用“分而治之”的方式，从大块到小块逐步交换：

1. 16 位交换：高 16 位与低 16 位交换。
2. 8 位交换：每个 16 位块内，高 8 位与低 8 位交换。
3. 4 位交换：每个 8 位块内，高 4 位与低 4 位交换。
4. 2 位交换：每个 4 位块内，高 2 位与低 2 位交换。
5. 1 位交换：每个 2 位块内，相邻 1 位交换。

掩码说明

0x0000FFFF : 低 16 位全 1

0x00FF00FF : 每 16 位中低 8 位全 1

0x0F0F0F0F : 每 8 位中低 4 位全 1

0x33333333 : 每 4 位中低 2 位全 1

0x55555555 : 每 2 位中低 1 位全 1

示例验证

以 $x = 0x80000004$ 为例：

- 经过 5 层交换后，结果为 0x20000001。

操作数统计：共使用 25 个运算符（5 次 >>，5 次 <<，10 次 &，5 次 |），符合最大操作数不超过 34 的限制。